

Robot URDF Model

Robot URDF Model

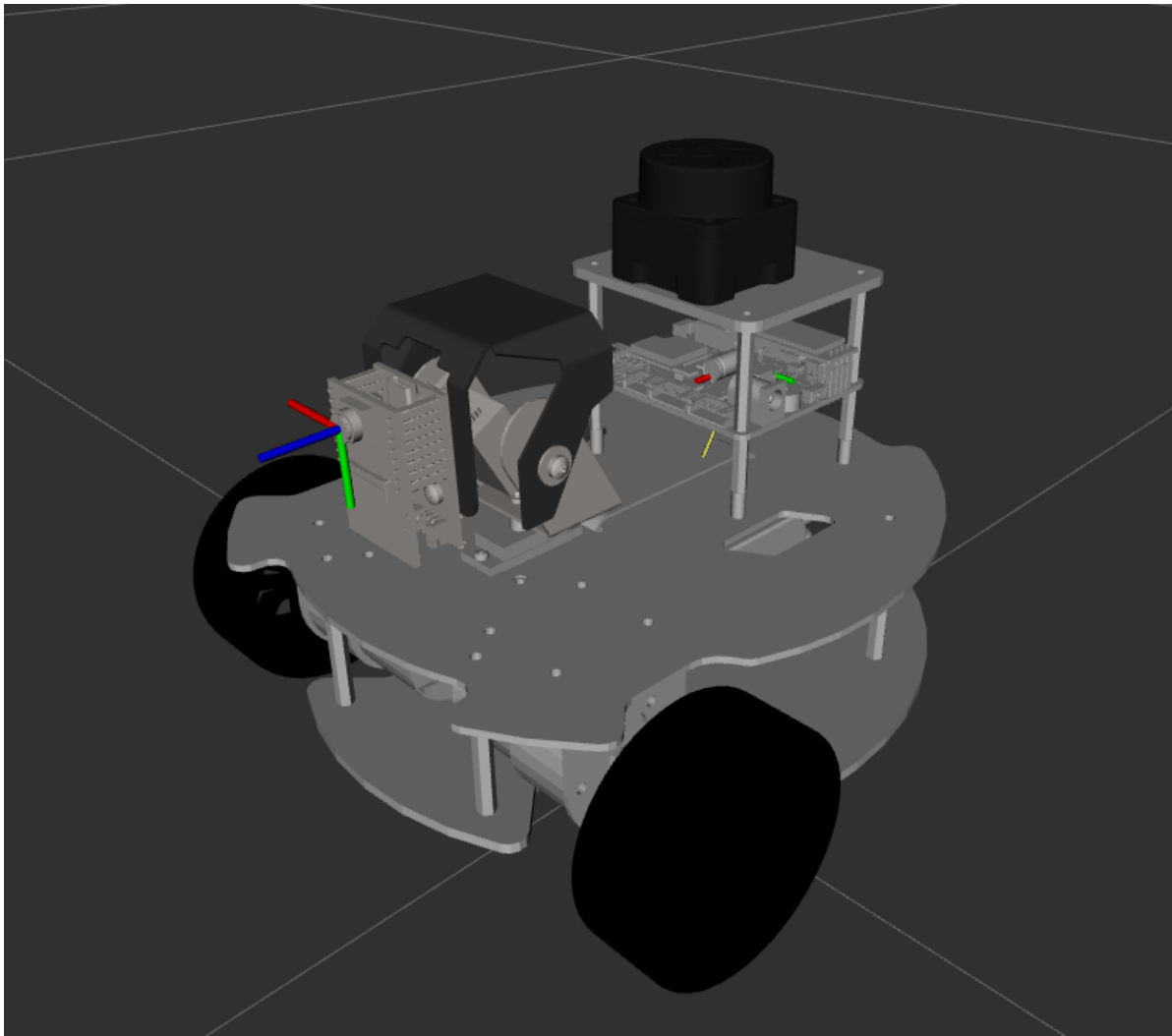
1. Course Content
2. Demonstration
 - Operating Instructions
3. Source Code Analysis
 - 3.1 model_display.py
 - 3.2 Model File

1. Course Content

- Start the 3D state estimation layer and view the car's URDF model in real time in RViz2 using the **Orbit 3D view**. The model pose is driven in real time by the IMU data of the ESP32 control board — pick up the car and flip or rotate it freely, and the 3D model in RViz2 follows the motion, giving you an intuitive understanding of the coordinate systems of each sensor and the joint structure.

2. Demonstration

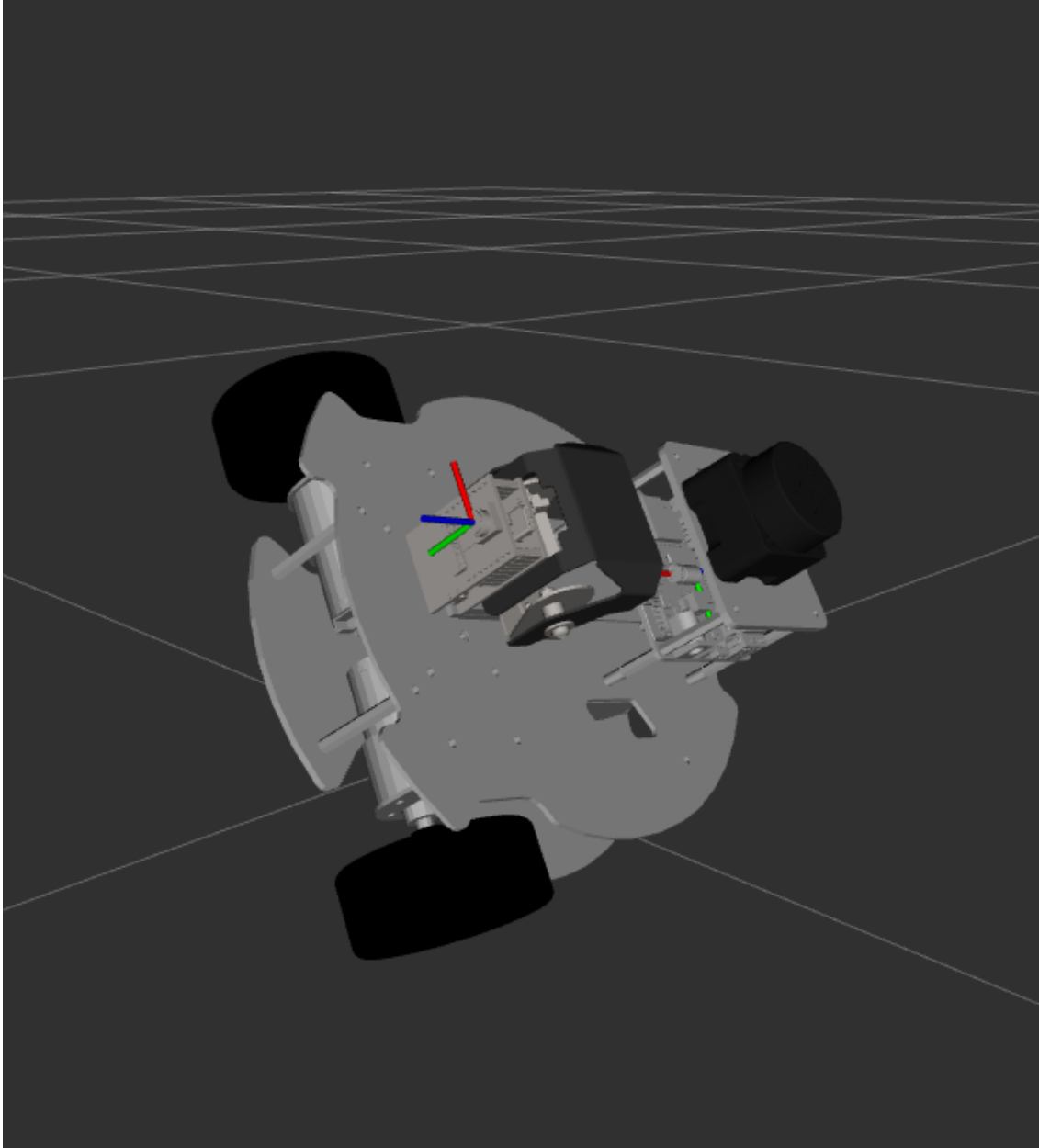
```
ros2 launch microros_v2_bringup display_model.launch.py
```



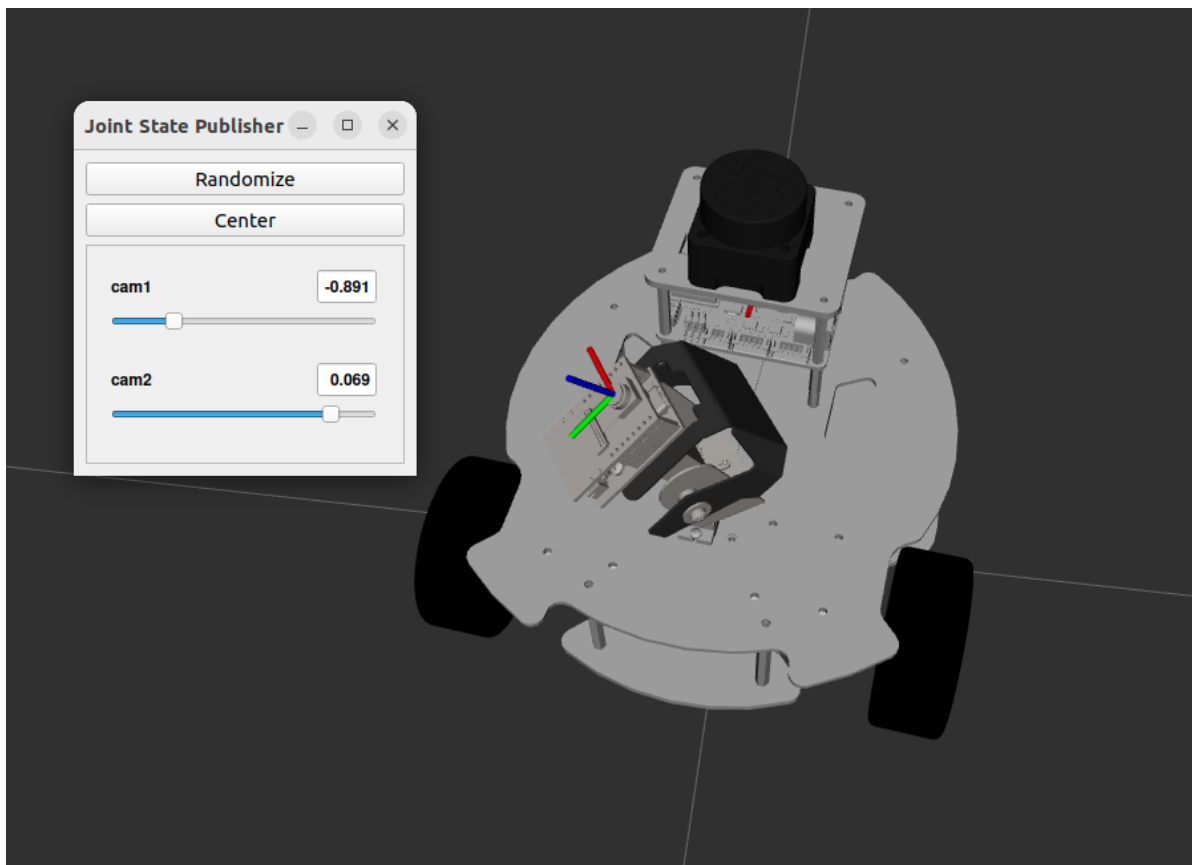
Operating Instructions

After startup, RViz2 displays the car's 3D model. Since the IMU drives it in real time, **pick up the car and tilt or flip it in your hand**, and the model in RViz2 follows the motion immediately:

- **Rotate the car:** The model flips synchronously in RViz2 (driven by the full IMU roll/pitch/yaw pose)



- Adjust the gimbal servo angles through the GUI sliders: the cam1/cam2 joints update the model display effect in real time



3. Source Code Analysis

3.1 model_display.py

- Source code path

```
$HOME/microros_ws/src/common_demo/common_demo/model_display/model_display.py
```

`model_display.py` converts the IMU real-time pose data into TF transforms, so that the URDF model in RViz2 flips and rotates together with the real car.

1. **Node initialization (`__init__`)**: Creates a `TransformBroadcaster` for publishing TF transforms; subscribes to the `/imu` topic with the `qos_profile_sensor_data` QoS (IMU usually uses best-effort QoS), with the callback function `imu_callback`.
2. **IMU callback (`imu_callback`)**: After receiving an IMU message, constructs a `TransformStamped`:
 - `header.frame_id = 'odom'`, `child_frame_id = 'base_footprint'`, establishing the `odom → base_footprint` transform chain
 - The translation is fixed at `(0, 0, 0.03)`, only to keep the model floating at the origin for easy observation of the pose; no position estimation is performed
 - The rotation quaternion is taken directly from `msg.orientation` (i.e., the roll/pitch/yaw obtained from IMU fusion), achieving real-time full-pose driving
 - Calls `sendTransform` to publish; after receiving the TF, `robot_state_publisher` automatically computes the poses of all child links (base_link, gimbal, camera, etc.) based on the joint relationships in the URDF

3. **Cooperation with joint_state_publisher:** The IMU only drives the overall pose of `base_footprint`; the cam1/cam2 gimbal joint angles are published separately by `joint_state_publisher` subscribing to `/ptz_status`. When the two are combined, the model in RViz2 can simultaneously present the complete effect of the body tilting + the gimbal rotating.

```
class ModelDisplayNode(Node):
    def __init__(self):
        super().__init__('model_display')
        self.tf_broadcaster = TransformBroadcaster(self)
        # IMU data is usually published with best-effort QoS
        self.imu_sub = self.create_subscription(
            Imu, '/imu', self.imu_callback, qos_profile_sensor_data)
        self.get_logger().info('model_display started: /imu -> tf(odom-
>base_link)')

    def imu_callback(self, msg: Imu):
        t = TransformStamped()
        t.header.stamp = self.get_clock().now().to_msg()
        t.header.frame_id = 'odom'
        t.child_frame_id = 'base_footprint'
        # Translation stays at origin, only attitude is visualized
        t.transform.translation.x = 0.0
        t.transform.translation.y = 0.0
        t.transform.translation.z = 0.03
        t.transform.rotation = msg.orientation
        self.tf_broadcaster.sendTransform(t)
```

3.2 Model File

- Model file path

```
$HOME/microros_ws/src/robot_description/urdf/microros_v2.urdf.xacro
```

```
<robot xmlns:xacro="http://www.ros.org/wiki/xacro"
  name="microros_v2">

  <xacro:include filename="$(find robot_description)/urdf/camera.urdf.xacro"/>

  <!-- camera_type: ptz | no_ptz | basic_car -->
  <xacro:arg name="camera_type" default="ptz"/>

  <!-- base_link mesh: no_ptz uses base_link_noptz.STL, ptz/basic_car use
base_link.STL -->
  <xacro:property name="base_link_mesh"
    value="'${'base_link_noptz.STL' if '${arg camera_type}' == 'no_ptz' else
'base_link.STL'}'"/>

  <!-- base_footprint -->
  <link
    name="base_footprint">
```

```

</link>
<joint
  name="basetofootprint"
  type="fixed">
  <origin
    xyz="0.0 0.0 0.02"
    rpy="0 0 0" />
  <parent
    link="base_footprint" />
  <child
    link="base_link" />
</joint>

<!-- car base -->
<link
  name="base_link">
  <inertial>
    <origin
      xyz="0.0125358449468065 6.2338964078765E-07 0.0329029674944082"
      rpy="0 0 0" />
    <mass
      value="0.549545262878645" />
    <inertia
      ixx="0.000494117990279825"
      ixy="7.47435050338857E-08"
      ixz="-5.82265935748093E-08"
      iyy="0.00056957019186456"
      iyz="-1.73647671262012E-07"
      izz="0.00100869032893883" />
    </inertial>
  <visual>
    <origin
      xyz="0 0 0"
      rpy="0 0 0" />
    <geometry>
      <mesh
        filename="package://robot_description/meshes/${base_link_mesh}" />
      </geometry>
    <material
      name="">
      <color
        rgba="0.752941176470588 0.752941176470588 0.752941176470588 1" />
      </material>
    </visual>
  <collision>
    <origin
      xyz="0 0 0"
      rpy="0 0 0" />
    <geometry>
      <mesh
        filename="package://robot_description/meshes/${base_link_mesh}" />
      </geometry>
    </collision>
  </link>

```

.....

- URDF, whose full name is Unified Robot Description Format, is a robot model file described in XML format

Tag Introduction

- link, a link, which can be thought of as a human arm
- joint, a joint, which can be thought of as a human elbow joint
- The relationship between link and joint: two links are connected through a joint. Imagine an arm where the forearm (link) and the upper arm (link) are connected through the elbow joint (joint).
- In the URDF description language, link is used to describe physical characteristics:
 - Describes the visual display, the `<visual>` tag.
 - Describes the collision properties, the `<collision>` tag.
 - Describes the physical inertia, the `<inertial>` tag (not commonly used).
- origin

Describes the pose information; the xyz attribute describes the coordinate position in the global environment, and the rpy attribute describes its own orientation.
- mass

Describes the mass of the link.
- inertia

Inertial reference frame. Due to the symmetry of the rotational inertia matrix, only the 6 upper-triangular elements ixx , ixy , ixz , iyy , iyz , izz are needed as attributes.
- geometry

The tag describes the shape; the mesh attribute is mainly used to load texture files, and the filename attribute is the file path of the texture.
- material

The tag describes the material; the name attribute is **required**, can be empty, and can be repeated. The rgba attribute in the `<color>` tag represents red, green, blue, and transparency, separated by spaces. The color range is [0-1].

joint describes the relationship between two links, and the motion position and velocity limits, as well as the kinematic and dynamic properties.

Joint types:

- fixed: Fixed joint. Motion is not allowed; it serves to connect.
- continuous: Continuous joint. Can rotate continuously, without rotation angle limits.
- revolute: Revolute joint. Similar to continuous, but with rotation angle limits.
- prismatic: Prismatic joint. Moves along an axis, with position limits.
- floating: Floating joint. Has six degrees of freedom, 3T3R.
- planar: Planar joint. Allows translation or rotation orthogonal to a plane.
- In the `<joint>` tag, the name attribute is **required**, describing the name of the joint, and it must be unique.
- In the `<joint>` tag, the type attribute corresponds to the six joint types.

